Network Slicing con Ryu e Mininet

Pamela Longo, M63001877

Corso di Laurea Magistrale in Ingegneria Informatica
Università degli Studi di Napoli Federico II

1 Introduzione

Questo progetto affronta il *network slicing* in una rete definita via software: la stessa infrastruttura fisica viene suddivisa in porzioni logiche (*slice*), ciascuna con le proprie caratteristiche di banda e con regole di accesso diverse, e il tutto è governato da un controller centrale. Il sistema è stato realizzato con un controller **Ryu** che parla **OpenFlow 1.3**, una rete emulata con **Mininet** e switch **Open vSwitch**.

Le due modalità richieste dalla traccia sono lo *slicing per topologia* (*topology slicing*), in cui la slice dipende da *chi* comunica, e lo *slicing per servizio* (*service slicing*), in cui dipende da *che cosa* si trasmette. Sopra queste due modalità sono state aggiunte tre estensioni: lo **slicing dinamico**, che riassegna la banda inutilizzata, una difesa contro il **MAC spoofing** e una **dashboard** di monitoraggio basata sui contatori reali della rete.

Tutto il lavoro è stato eseguito e misurato in una macchina virtuale Ubuntu 22.04 (Mininet 2.3, Open vSwitch, Ryu 4.34, iperf, tcpdump). In questa relazione vengono descritti prima il contesto teorico, poi le scelte di progetto, quindi ciascuna funzionalità con il relativo esperimento e i risultati misurati, infine i limiti e i possibili sviluppi.

2 Contesto teorico

2.1 SDN e OpenFlow

In una rete tradizionale ogni apparato decide da solo come inoltrare i pacchetti. Nelle *Software-Defined Network* il *control plane* (le decisioni) è separato dal *data plane* (l'inoltro) e affidato a un controller centrale, che programma gli switch tramite un protocollo, qui OpenFlow. Uno switch OpenFlow contiene una *flow table*: ogni regola ha un *match* (porta di ingresso, MAC, IP, protocollo, porta di livello 4, ...), una *priorità*, delle *azioni* (inoltra su una porta, scarta, assegna a una coda, invia al controller), dei *contatori* di pacchetti e byte, dei *timeout* (*idle*: scade se non passa traffico; *hard*: scade comunque) e un *cookie*, cioè un'etichetta scelta dal controller. Se un pacchetto non corrisponde a nessuna regola vale la regola di *table-miss*, che di solito lo manda al controller in un messaggio *Packet-In*; il controller risponde con un *Flow-Mod* (installa o cancella una regola) e, per non perdere il primo pacchetto, con un *Packet-Out*.

2.2 Network slicing

Il *network slicing* divide la rete in reti virtuali isolate sopra la stessa infrastruttura. Nel *topology slicing* una slice è definita da un insieme di nodi e collegamenti: i pacchetti di una coppia di host possono percorrere solo il cammino della propria slice, e gli host di slice diverse non si raggiungono. Nel *service slicing* la slice è scelta dal tipo di traffico, riconosciuto di solito dal protocollo e dalla porta di livello 4, e a ogni slice si associa un trattamento (per esempio una quantità di banda garantita).

Il vantaggio di SDN è che entrambe le politiche stanno in un unico programma, il controller, che può anche cambiarle mentre la rete funziona.

2.3 Code e QoS con Open vSwitch

Quando un collegamento è saturo i pacchetti si accodano e, se la coda si riempie, vengono scartati: un flusso video ne soffre. Le *code* di Open vSwitch (disciplina `linux-htb`) permettono di fissare per ogni porta un tetto di banda e, per ciascuna coda, un minimo garantito (`min-rate`) e un massimo (`max-rate`). Il controller sceglie la coda di un flusso con l'azione OpenFlow `SetQueue`. È la stessa meccanica con cui sono stati imposti i limiti di 10 e 1 Mbps alle due slice.

3 La rete e le scelte di progetto

3.1 Topologia

La rete ha quattro host (`h1`,`h2` collegati a `s1`; `h3`,`h4` collegati a `s4`) e due percorsi disgiunti tra `s1` e `s4`, che costituiscono le due slice. I numeri di porta sono fissati nel codice, quindi il controller conosce la rete a priori.

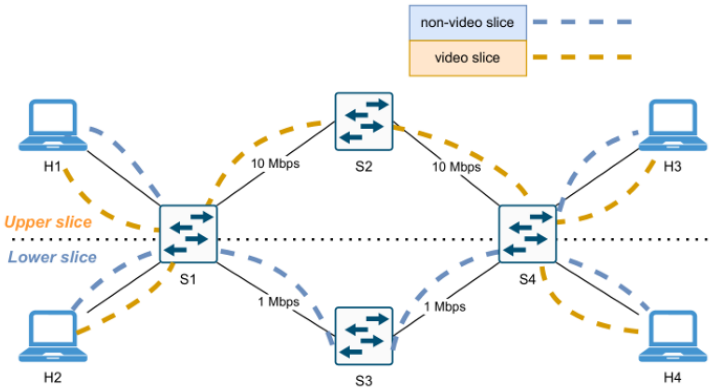


Figura 1: Rete a rombo con le due slice.

Slice	Percorso	Capacità	Uso
alta	`s1-s2-s4`	10 Mbps	video, traffico critico
bassa	`s1-s3-s4`	1 Mbps	traffico normale (best effort)

```
19  HOSTS = {
20      "h1": Host("h1", "00:00:00:00:00:01", "10.0.0.1", 1, 1),
21      "h2": Host("h2", "00:00:00:00:00:02", "10.0.0.2", 1, 2),
22      "h3": Host("h3", "00:00:00:00:00:03", "10.0.0.3", 4, 1),
23      "h4": Host("h4", "00:00:00:00:00:04", "10.0.0.4", 4, 2),
24  }
```

(a) Host: MAC, IP, switch e porta (`slice_topology.py`, righe 19–24).

```
40  # capacita' dei link (Mbps) e regole delle code
41  LINK_MBPS = {UPPER: 10, LOWER: 1}
42  QUEUE_PREMIUM, QUEUE_STANDARD = 1, 2
43  QUEUE_MIN_MBPS = {UPPER: {QUEUE_PREMIUM: 8, QUEUE_STANDARD: 1},
44                    LOWER: {QUEUE_PREMIUM: 0.5, QUEUE_STANDARD: 0.5}}
45
46  # coppie ammesse nel topology slicing e relativa slice
47  PAIR_SLICE = {frozenset(("h1", "h3")): UPPER, frozenset(("h2", "h4")): LOWER}
```

(b) Capacità, code e coppie ammesse (`slice_topology.py`, righe 40–47).

Figura 2: La topologia nel codice: i dati statici della rete.

```
55      for dpid, port, sl in T.core_egress_ports():
56          rate = int(T.LINK_MBPS[sl] * 1_000_000)
57          mins = T.QUEUE_MIN_MBPS[sl]
58          _vsctl("--", "set", "port", "s%d-eth%d" % (dpid, port), "qos=@q",
59                 "--", "--id=@q", "create", "qos", "type=linux-htb",
60                 "other-config:max-rate=%d" % rate,
61                 "queues:%d=@q1" % T.QUEUE_PREMIUM, "queues:%d=@q2" % T.QUEUE_STANDARD,
62                 "--", "--id=@q1", "create", "queue",
63                 "other-config:min-rate=%d" % int(mins[T.QUEUE_PREMIUM] * 1e6),
64                 "other-config:max-rate=%d" % rate,
65                 "--", "--id=@q2", "create", "queue",
66                 "other-config:min-rate=%d" % int(mins[T.QUEUE_STANDARD] * 1e6),
67                 "other-config:max-rate=%d" % rate)
```

Figura 3: Configurazione delle code HTB con `ovs-vsctl`: `max-rate` pari alla capacità del link, `min-rate` per la coda premium e per quella standard (`topology.py`, righe 55–67).

3.2 Il problema del ciclo: proxy ARP e `failMode=secure`

La rete a rombo contiene un ciclo, e questo crea un problema concreto con l'ARP. Una richiesta ARP ("chi ha questo indirizzo IP?") è un *broadcast*: se gli switch la inoltrassero in flooding, girerebbe nel ciclo all'infinito (*broadcast storm*). Poiché IP e MAC degli host sono noti, si è scelto di far rispondere **il controller**: alla richiesta ARP risponde con un Packet-Out sulla sola porta di chi ha chiesto, e nella rete non circola alcun broadcast. Per lo stesso motivo gli switch sono configurati con `failMode=secure`: di default Open vSwitch, se perde il controller, si comporta come un normale switch L2 e farebbe flooding, innescando la tempesta; in modalità *secure* invece smette semplicemente di inoltrare finché il controller non torna.

```
26      for dpid in (1, 2, 3, 4):
27          # failMode=secure: se cade il controller lo switch NON diventa un L2 normale
28          # (nel rombo ci sarebbe un loop di broadcast)
29          sw[dpid] = net.addSwitch("s%d" % dpid, protocols="OpenFlow13",
30                                   dpid="%016x" % dpid, failMode="secure")
31      for h in T.HOSTS.values():
32          host = net.addHost(h.name, ip=h.ip + "/24", mac=h.mac)
33          net.addLink(host, sw[h.dpid], port1=0, port2=h.port, bw=100)
34      for a, b in T.SWITCH_LINKS:
35          net.addLink(sw[a], sw[b], port1=T.CORE_PORT[(a, b)], port2=T.CORE_PORT[(b, a)])
```

Figura 4: Creazione della rete in Mininet: gli switch hanno `failMode="secure"`, poi vengono aggiunti host e collegamenti (`topology.py`, righe 26–35).

3.3 Architettura del software

Il codice è diviso in moduli con compiti distinti, in modo da separare le decisioni dal codice OpenFlow.

Modulo	Ruolo
`slice_topology.py`	dati statici della rete: host, MAC, IP, porte, capacità dei link
`slice_config.py`	parametri (modalità, timeout, priorità), modificabili con variabili d'ambiente
`slice_policy.py`	**decisioni** senza OpenFlow: ammissibilità di una coppia, classificazione, percorso, anti-spoofing, stato dello slicing dinamico
`slice_controller.py`	applicazione Ryu: Packet-In, installazione delle regole, monitoraggio, stato per la dashboard

3.4 Il ciclo di vita di un pacchetto

Il controller è *ibrido*: reattivo per i flussi (decide al primo pacchetto) e proattivo per le regole di sicurezza (le installa alla connessione degli switch). Si consideri un flusso video UDP da `h1` a `h3`:

1. `h1` invia una richiesta ARP per il MAC di `h3`; arriva a `s1`, che non ha regole e la manda al controller (Packet-In).
2. Il controller verifica l'anti-spoofing (MAC e IP del pacchetto coincidono con quelli legati alla porta?), poi risponde lui all'ARP.
3. `h1` manda il primo pacchetto UDP verso la porta 9999: nessuna regola, di nuovo Packet-In.
4. Il controller riconosce il flusso (UDP/9999 $\Rightarrow$ *video*), sceglie la slice alta e la coda premium e calcola il percorso: `s1` (porta 1 $\rightarrow$ 3), `s2` (1 $\rightarrow$ 2), `s4` (3 $\rightarrow$ 1).
5. Installa **una regola su ciascuno dei tre switch** (Flow-Mod), con `in_port`, MAC sorgente e destinazione, protocollo e porta nel match, e come azione "metti in coda 1, poi esci dalla porta X".
6. Rimanda il primo pacchetto con un Packet-Out, così non si perde.
7. I pacchetti successivi trovano la regola e non disturbano più il controller; la regola scade dopo 10 s di inattività.

La scelta di installare la regola su *tutti* gli switch del percorso evita che ogni salto generi un suo Packet-In e che il pacchetto arrivi ai nodi centrali senza istruzioni.

```
169         # 1) anti-spoofing: prima di qualsiasi altra cosa
170         if self.cfg.SPOOF_MODE != "off":
171             reason = self.guard.check(dp.id, in_port, eth.src, claimed_ip)
172             if reason is not None:
173                 self._on_spoof(dp, in_port, eth.src, reason)
174                 if self.cfg.SPOOF_MODE == "enforce":
175                     return
176
177         # 2) ARP e IP
178         if arp_pkt is not None:
179             self._handle_arp(dp, in_port, eth, arp_pkt)
180         elif ip_pkt is not None:
181             self._handle_ip(dp, in_port, msg.data, eth, pkt, ip_pkt)
```

Figura 5: Gestione del Packet-In: prima il controllo anti-spoofing, poi lo smistamento tra ARP e IP (`slice_controller.py`, righe 169–181).

3.5 Regole, priorità e cookie

Alla connessione ogni switch riceve la regola di table-miss e un piccolo insieme di regole statiche: sugli switch centrali (`s2`, `s3`) "ciò che entra da una porta esce dall'altra"; sugli switch di bordo "ciò che arriva dal core ed è destinato a un host locale esce sulla porta di quell'host". Sono una rete di sicurezza: se la regola di un flusso scade o viene cancellata a metà percorso, il pacchetto prosegue lungo la slice in cui è entrato. In modalità service e dynamic ci sono inoltre regole proattive che ammettono sempre il `ping`, legate a porta, MAC *e* IP dell'host vero, che instradano l'ICMP sulla slice bassa; nel topology slicing queste regole non esistono, perché un ping tra coppie vietate deve fallire.

```
62      # ---------------------------------------------------------------- switch
63      @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
64      def _features(self, ev):
65          dp = ev.msg.datapath
66          ofp, parser = dp.ofproto, dp.ofproto_parser
67          # table-miss: tutto cio' che non ha una regola va al controller
68          actions = [parser.OFPActionOutput(ofp.OFPP_CONTROLLER, ofp.OFPCML_NO_BUFFER)]
69          self._flow_mod(dp, 0, parser.OFPMatch(), actions)
70          self._install_static(dp)
```

Figura 6: Alla connessione di uno switch: regola di table-miss (priorità 0, verso il controller) e installazione delle regole statiche (`slice_controller.py`, righe 63–70).

Priorità	Regola
0	table-miss: manda al controller
5	statiche: inoltro sul core, consegna agli host dal core
10	inoltro per flusso (decisioni del controller)
50	drop di coppie non autorizzate (topology slicing)
60	ping sempre ammessi (service e dynamic)
90	drop di MAC falsificati: vince su tutto

Le priorità sono ordinate in modo che le decisioni di sicurezza prevalgano su quelle di servizio. In particolare le regole ICMP (60) stanno *sotto* il drop anti-spoofing (90) e sono legate anche all'IP dell'host vero: una priorità più alta le renderebbe un varco per chi falsifica il MAC. Ogni regola porta un **cookie** che ne indica la classe (1 video, 2 normale, 3 normale in prestito, 4 statica): così, per riassegnare tutti i flussi normali, basta un solo messaggio di cancellazione per cookie.

4 Topology slicing

Obiettivo. Solo `h1`↔`h3` (slice alta) e `h2`↔`h4` (slice bassa) possono comunicare; ogni altra coppia, ARP compreso, è bloccata. **Implementazione.** La policy contiene la tabella delle coppie ammesse e della loro slice; per le coppie non ammesse il controller installa un drop a priorità 50 e non risponde all'ARP.

Esperimento. `run_experiment.py -mode topology` prova tutti i ping tra gli host, legge i contatori dei pacchetti entrati in `s2` e `s3` e cattura con `tcpdump` i ping sui collegamenti `s2-eth1` e `s3-eth1`.

Risultati. Risultano raggiungibili solo le quattro direzioni ammesse. I pacchetti h1–h3 entrano in `s2` (8) e non in `s3` (0); quelli h2–h4 il contrario (0 e 8). Il `tcpdump` mostra 5 echo request h1→h3 su `s2-eth1` e nessuno su `s3-eth1`, e viceversa per h2→h4. Il `tcpdump` è una misura *indipendente* dal controller: mostra i pacchetti sul collegamento, non ciò che il controller dichiara di aver deciso. Le regole installate (`dump-flows`) mostrano `in_port`, MAC e porta di uscita verso la slice giusta.

```
pamela@pamela-VirtualBox:~/Scrivania/NCIS/sdn-slicing-project$    sudo python3 run_experiment.py --mode topology
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
== modalita': topology ==
Raggiungibilita' (ping):
  h1 -> h2 : bloccato  atteso: vietato
  h1 -> h3 : raggiungibile atteso: ok
  h1 -> h4 : bloccato  atteso: vietato
  h2 -> h1 : bloccato  atteso: vietato
  h2 -> h3 : bloccato  atteso: vietato
  h2 -> h4 : raggiungibile atteso: ok
  h3 -> h1 : raggiungibile atteso: ok
  h3 -> h2 : bloccato  atteso: vietato
  h3 -> h4 : bloccato  atteso: vietato
  h4 -> h1 : bloccato  atteso: vietato
  h4 -> h2 : raggiungibile atteso: ok
  h4 -> h3 : bloccato  atteso: vietato
Percorso h1-h3: pacchetti entrati in s2=8, in s3=0  (atteso slice ALTA (s2))
Percorso h2-h4: pacchetti entrati in s2=0, in s3=8  (atteso slice BASSA (s3))
Verifica con tcpdump (file tcpdump_*.txt):
  ping h1->h3: echo request visti su s2-eth1 = 5, su s3-eth1 = 0  (atteso: solo s2 (slice alta))
  ping h2->h4: echo request visti su s2-eth1 = 0, su s3-eth1 = 5  (atteso: solo s3 (slice bassa))
Regole di tutti gli switch salvate in flows_s1..s4.txt
ESITO: ok
pamela@pamela-VirtualBox:~/Scrivania/NCIS/sdn-slicing-project$
```

Figura 7: Topology slicing: raggiungibilità e percorsi (esito di `run_experiment.py --mode topology`). Le righe `sch_htb` in alto sono avvisi del kernel sulla configurazione delle code HTB e non influenzano il risultato.

Apri — tcpdump_h1_h3_s2.txt [Sola lettura] — ~/Scrivania/NCIS/sdn-slicing-project/results/topology — Salva

```
 1 23:54:00.064981 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 55753, seq 1, length 64
 2 23:54:00.065019 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 55753, seq 1, length 64
 3 23:54:00.391097 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 55753, seq 2, length 64
 4 23:54:00.391136 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 55753, seq 2, length 64
 5 23:54:00.770527 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 55753, seq 3, length 64
 6 23:54:00.770593 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 55753, seq 3, length 64
 7 23:54:01.109892 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 55753, seq 4, length 64
 8 23:54:01.110042 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 55753, seq 4, length 64
 9 23:54:01.423350 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 55753, seq 5, length 64
10 23:54:01.423388 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 55753, seq 5, length 64
11
```

Apri — tcpdump_h2_h4_s3.txt [Sola lettura] — ~/Scrivania/NCIS/sdn-slicing-project/results/topology — Salva

```
 1 23:54:10.333935 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 3775, seq 1, length 64
 2 23:54:10.333973 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 3775, seq 1, length 64
 3 23:54:10.638835 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 3775, seq 2, length 64
 4 23:54:10.638873 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 3775, seq 2, length 64
 5 23:54:10.948995 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 3775, seq 3, length 64
 6 23:54:10.949032 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 3775, seq 3, length 64
 7 23:54:11.274815 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 3775, seq 4, length 64
 8 23:54:11.274854 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 3775, seq 4, length 64
 9 23:54:11.600326 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 3775, seq 5, length 64
10 23:54:11.600365 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 3775, seq 5, length 64
11
```

Figura 8: Topology slicing: verifica con `tcpdump`. Gli echo request di h1–h3 compaiono solo su `s2-eth1`, quelli di h2–h4 solo su `s3-eth1`.

Apri — flows_s1.txt [Sola lettura] — ~/Scrivania/NCIS/sdn-slicing-project/results/topology — Salva

```
1 OFPST_FLOW reply (OF1.3) (xid=0x2):
2  cookie=0x2, duration=35.388s, table=0, n_packets=21, n_bytes=2058, idle_timeout=10,
  priority=10,icmp,in_port=1,dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:03 actions=output:
  3
3  cookie=0x2, duration=35.378s, table=0, n_packets=22, n_bytes=2156, idle_timeout=10,
  priority=10,icmp,in_port=3,dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01 actions=output:
  1
4  cookie=0x2, duration=28.221s, table=0, n_packets=21, n_bytes=2058, idle_timeout=10,
  priority=10,icmp,in_port=2,dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:04 actions=output:
  4
5  cookie=0x2, duration=28.213s, table=0, n_packets=22, n_bytes=2156, idle_timeout=10,
  priority=10,icmp,in_port=4,dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:02 actions=output:
  2
```

Figura 9: Topology slicing: regole installate su `s1`. Le righe evidenziate mostrano che h1–h3 esce dalla porta 3 (verso `s2`, slice alta) e h2–h4 dalla porta 4 (verso `s3`, slice bassa).

5 Service slicing

Obiettivo. Tutti gli host si raggiungono, ma il trattamento dipende dal servizio: il video (UDP, porta di destinazione 9999) viaggia sulla slice alta con la coda premium; tutto il resto sulla slice bassa con la coda standard. **Implementazione.** Al primo pacchetto di un flusso il controller estrae protocollo e porta di livello 4 e li passa alla policy, che restituisce slice, coda e cookie. Il comportamento è reso visibile da una riga di log per flusso, per esempio `Traffic Identified: [VIDEO] -> [PREMIUM] h1 -> h3 UDP/9999 slice alta`.

```
48      def is_video(self, proto, dport):
49          return proto == UDP and dport == self.cfg.VIDEO_PORT
50
51      def decide(self, src, dst, proto, dport):
52          if not self.allowed(src, dst):
53              return Decision(False, None, None, 0, "coppia non autorizzata")
54          video = self.mode != "topology" and self.is_video(proto, dport)
55
56          if self.mode == "topology":
57              sl = T.PAIR_SLICE[frozenset((src, dst))]
58          elif video:
59              sl = T.UPPER
60          elif self.mode == "dynamic" and not self.state.video_active:
61              sl = T.UPPER                        # la slice video e' libera: la presto
62          else:
63              sl = T.LOWER
```

Figura 10: Classificazione del traffico: `is_video()` riconosce il video (UDP, porta 9999) e `decide()` sceglie la slice (`slice_policy.py`, righe 48–63).

```
249         fields = dict(in_port=hop.in_port, eth_type=ether_types.ETH_TYPE_IP,
250                       eth_src=eth.src, eth_dst=eth.dst, ip_proto=proto)
251         if dport and proto == P.UDP:
252             fields["udp_dst"] = dport
253         elif dport and proto == P.TCP:
254             fields["tcp_dst"] = dport
255         actions = []
256         if dec.queue is not None and hop.core:
257             actions.append(parser.OFPActionSetQueue(dec.queue))
258         actions.append(parser.OFPActionOutput(hop.out_port))
259         self._flow_mod(dp, self.cfg.FLOW_PRIORITY, parser.OFPMatch(**fields), actions,
260                        cookie=dec.cookie, idle=self.cfg.FLOW_IDLE_TIMEOUT)
```

Figura 11: Installazione della regola su ciascuno switch del percorso: match su porta, MAC, protocollo e porta L4, azione `OFPActionSetQueue` verso la coda scelta, cookie e `idle_timeout` (`slice_controller.py`, righe 249–260).

Esperimento. Un video UDP da 8 Mbps (`h1`→`h3`, porta 9999) e un TCP normale (`h2`→`h4`) da soli e insieme. Le misure sono prese lato *server* iperf (cioè ciò che arriva davvero) e si aspetta che la coda della slice bassa si svuoti tra una prova e l'altra, leggendo i contatori `tc`. L'esperimento è stato ripetuto con lo stesso esito.

Prova	Video (Mbps)	TCP (Mbps)
video da solo	8,36 – 8,39	–
TCP da solo	–	0,90 – 0,95
video + TCP insieme	8,39	0,96

Discussione. Il video riceve la banda offerta anche quando c'è il TCP, e il TCP resta circa a 1 Mbps, il limite della slice bassa, con o senza video: i due servizi non si influenzano, che è esattamente lo scopo dell'isolamento. Il dump delle regole di `s1` durante la prova mostra la causa: la regola del video porta `set_queue:1,output:3` (coda premium verso `s2`), quella del TCP `set_queue:2,output:4` (coda standard verso `s3`).

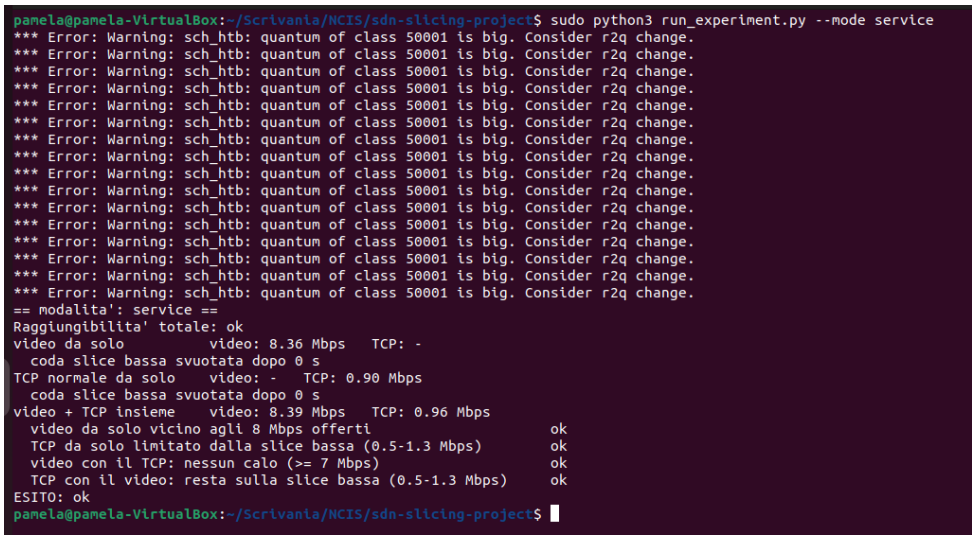


Figura 12: Service slicing: esito di `run_experiment.py --mode service`, con le tre prove (video da solo, TCP da solo, video e TCP insieme) e i controlli `ok`. Le righe `sch_htb` in alto sono avvisi del kernel sulla configurazione delle code HTB e non influenzano il risultato.

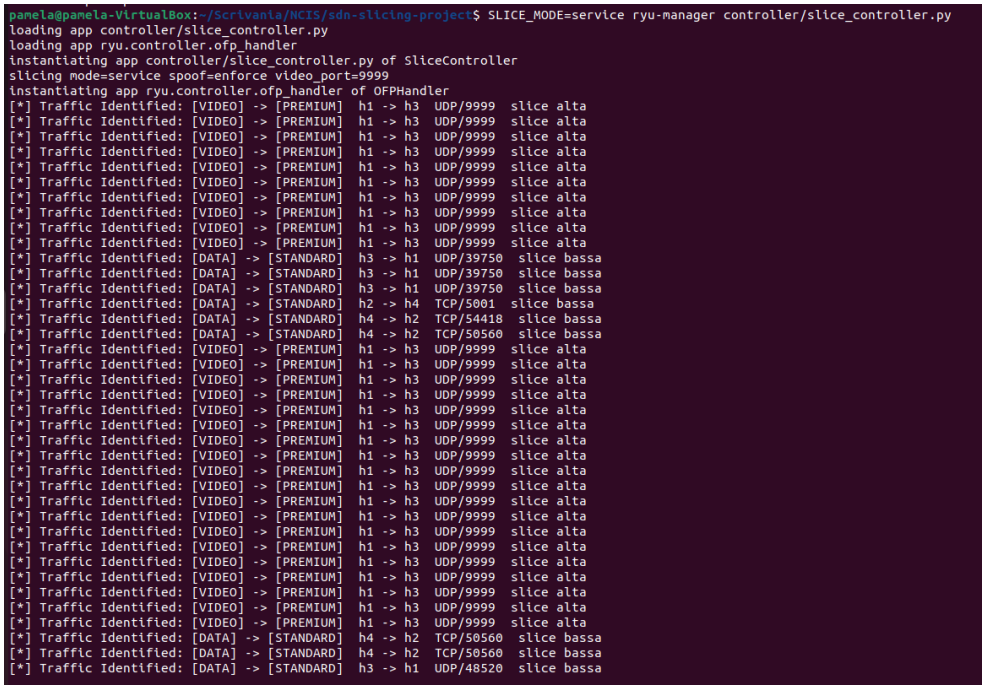


Figura 13: Service slicing: log del controller. Il video (UDP/9999) è classificato `VIDEO` → `PREMIUM` sulla slice alta, il resto `DATA` → `STANDARD` sulla slice bassa.

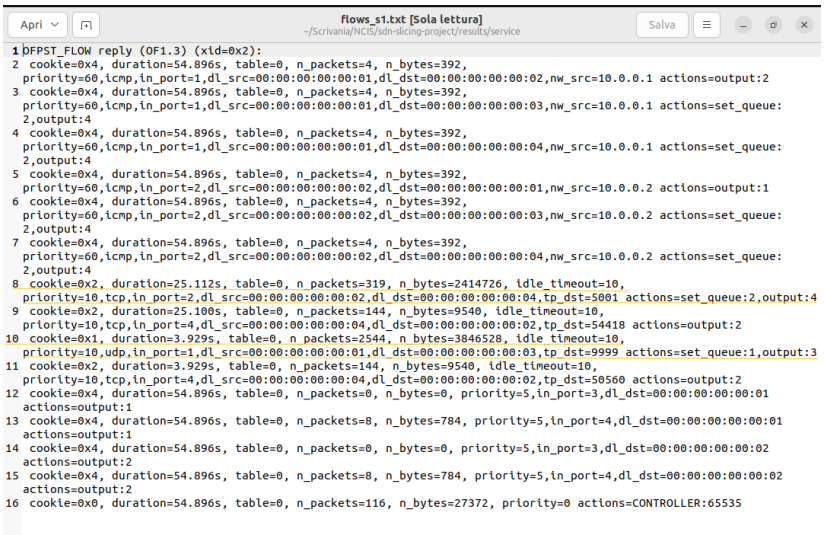


Figura 14: Service slicing: regole installate su `s1` durante la prova con video e TCP insieme. La regola del video (riga con `tp_dst=9999`) usa la coda 1, quella del TCP (`tp_dst=5001`) la coda 2.

Per leggere una regola, per esempio

```
priority=10,udp,in_port=1,dl_src=..:01,dl_dst=..:03,tp_dst=9999
  actions=set_queue:1,output:3
```

significa: “i pacchetti UDP/9999 da `h1` a `h3` entrati dalla porta 1 vanno nella coda premium e escono dalla porta 3 (verso `s2`, slice alta)”.

6 Slicing dinamico

Motivazione. Con il video fermo la slice alta (10 Mbps) è sprecata mentre il traffico normale è confinato in 1 Mbps. Lo slicing dinamico la presta al traffico normale e la riprende quando il video riparte. Non si tratta di una terza slice: le slice restano due e cambia solo la regola per il traffico normale (da slice bassa a slice alta e viceversa).

Algoritmo. Un thread del controller chiede ogni secondo le statistiche di tutti i flussi e guarda i byte dei flussi con cookie 1 (video). Se aumentano, il video è attivo; se per 3 s non aumentano, la slice alta è libera. Il primo Packet-In di un video imposta lo stato “attivo” immediatamente. Ad ogni cambio di stato i flussi normali interessati vengono cancellati per cookie e tornano al controller, che li reinstalla sulla slice giusta (cookie 3 quando sono “in prestito”).

```
133     def observe(self, now, new_video_bytes):
134         if new_video_bytes > 0:
135             self.last_video = now
136         return self._refresh(now)
137
138     def _refresh(self, now):
139         active = self.last_video is not None and (now - self.last_video) < self.idle_s
140         changed = active != self.video_active
141         self.video_active = active
142         return changed
```

Figura 15: Rilevazione dello stato del video: `observe()` aggiorna l’istante dell’ultimo video visto e `_refresh()` dichiara il video fermo dopo 3 s di silenzio (`slice_policy.py`, righe 133–142).

```
292     # ------------------------------------------------------ slicing dinamico
293     def _on_video_state(self, active):
294         """Il video e' partito o si e' fermato: i flussi normali vanno riassegnati."""
295         self.counters["reroutes"] += 1
296         if active:
297             self._delete_by_cookie(P.COOKIE_BORROW)    # chi stava in prestito torna in basso
298             self._event("dinamico", "video attivo: il traffico normale lascia la slice alta")
299         else:
300             self._delete_by_cookie(P.COOKIE_NORMAL)    # puo' salire sulla slice alta libera
301             self._event("dinamico", "video fermo: il traffico normale puo' usare la slice alta")
```

Figura 16: Riassegnazione dei flussi al cambio di stato: con il video attivo si cancellano per cookie i flussi in prestito (cookie 3), con il video fermo quelli normali (cookie 2), che tornano al controller e vengono reinstallati sulla slice giusta (`slice_controller.py`, righe 293–301).

Esperimento. Il TCP normale parte da solo; a 9 s parte il video per 9 s; poi termina.

Fase	TCP normale (Mbps)	Atteso
prima del video	9,55	≈ 9–10 (slice alta)
durante il video	0,84	≈ 1 (slice bassa)
dopo il video	8,19 (media)	≈ 9–10 (slice alta)

Il video resta a 8,41 Mbps. **Discussione.** Il comportamento è quello previsto: il TCP usa la slice alta finché è libera, scende a circa 1 Mbps quando arriva il video e risale quando il video finisce. Il passaggio non è istantaneo: il TCP risale circa 4–5 s dopo la fine del video, per tre motivi che si sommano (3 s di silenzio prima di dichiarare il video fermo, polling a 1 s, tempo per reinstallare le regole). È un compromesso: una soglia di silenzio più corta reagirebbe prima, ma rischierebbe di scambiare una breve pausa del video per la sua fine.

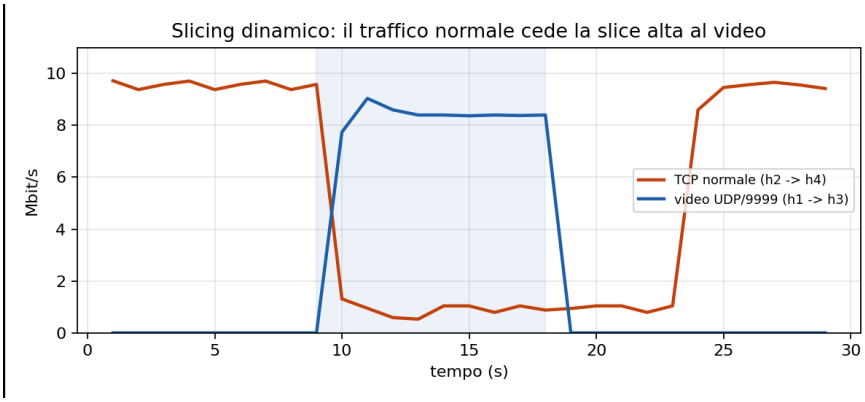


Figura 17: Slicing dinamico: il traffico normale cede la slice alta al video e la riprende.

7 Anti-spoofing

La minaccia. Il controller riconosce gli host dai MAC. Se un host (qui `h2`) dichiara il MAC di un altro (`h1`) può essere trattato come lui e ottenere il suo trattamento, o farsi passare per lui. **La difesa.** Ogni porta di accesso è legata a un MAC e a un IP. Ogni pacchetto (ARP o IP) che arriva su una porta di accesso è controllato *prima di tutto il resto*. Se non corrisponde, è scartato, si registra un evento e si installa un drop a tempo (priorità 90, 30 s) con match su `in_port` e MAC dichiarato: l'host legittimo, che sta su un'altra porta, non è toccato. Gli eventi sono limitati a uno ogni 5 s per coppia (porta, MAC), così l'attaccante non può riempire il log o la tabella delle regole. La difesa è possibile perché `in_port` è nel match di ogni regola: ciò che identifica il traffico è anche da dove entra.

```
104     def check(self, dpid, port, mac, ip=None):
105         """None se va bene, altrimenti la ragione della violazione."""
106         bound = self.bindings.get((dpid, port))
107         if bound is None:
108             return None
109         if mac != bound[0]:
110             return "MAC %s sulla porta %d di s%d, legata a %s" % (mac, port, dpid, bound[0])
111         if ip and ip != "0.0.0.0" and ip != bound[1]:
112             return "IP %s sulla porta %d di s%d, legata a %s" % (ip, port, dpid, bound[1])
113         return None
```

Figura 18: Controllo anti-spoofing: `PortGuard.check()` confronta MAC e IP del pacchetto con quelli legati alla porta di accesso e restituisce il motivo della violazione, oppure `None` se tutto coincide (`slice_policy.py`, righe 104–113).

I due attacchi. `h2` si spaccia per `h1` verso `h3` in due modi: (1) cambia il MAC della propria interfaccia con `ip link`; (2) tiene il proprio MAC ma invia frame contraffatti costruiti con Scapy. Ogni prova è un invio UDP da 1 Mbps: 3 s per l'host legittimo, 5 s per l'attaccante.

	Senza difesa		Con difesa	
Byte arrivati a `h3`	`ip link`	Scapy	`ip link`	Scapy
h2 (attaccante)	625000	370000	0	0
h1 (legittimo)	375000	375000	375000	375000

Discussione. Senza difesa entrambi gli attacchi passano: l'attaccante manda 625000 byte (1 Mbps per 5 s) con `ip link` e 370000 con Scapy (meno, perché lo script costruisce ogni frame in Python e non mantiene esattamente 1 Mbps). Con la difesa nessun byte dell'attaccante arriva a `h3`, mentre il traffico legittimo (375000 byte, cioè 1 Mbps per 3 s) arriva per intero. Nel log compare una riga per attacco, per esempio "MAC ...:01 sulla porta 2 di s1, legata a ...:02 → scartato": un pacchetto con il MAC di `h1` è arrivato dalla porta di `h2`.

```
pamela@pamela-VirtualBox:~/Scrivania/NCIS/sdn-slicing-project$ sudo python3 spoof_demo.py --label senza_difesa
[sudo] password di pamela:
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
*** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
Attacco ip link (cambio MAC dell'interfaccia)
  h1 (legittimo)  byte arrivati a h3: 375000
  h2 (MAC di h1)  byte arrivati a h3: 625000
  ESITO: attacco NON bloccato
Attacco Scapy (frame contraffatti)
  h1 (legittimo)  byte arrivati a h3: 375000
  h2 (MAC di h1)  byte arrivati a h3: 370000
  ESITO: attacco NON bloccato
pamela@pamela-VirtualBox:~/Scrivania/NCIS/sdn-slicing-project$
```

Figura 19: MAC spoofing senza difesa: con `SLICE_SPOOF_MODE=off` entrambi gli attacchi (`ip link` e Scapy) passano, e h2 riesce a far arrivare byte a h3 con il MAC di h1. Le righe `sch_htb` in alto sono avvisi del kernel sulla configurazione delle code HTB e non influenzano il risultato.

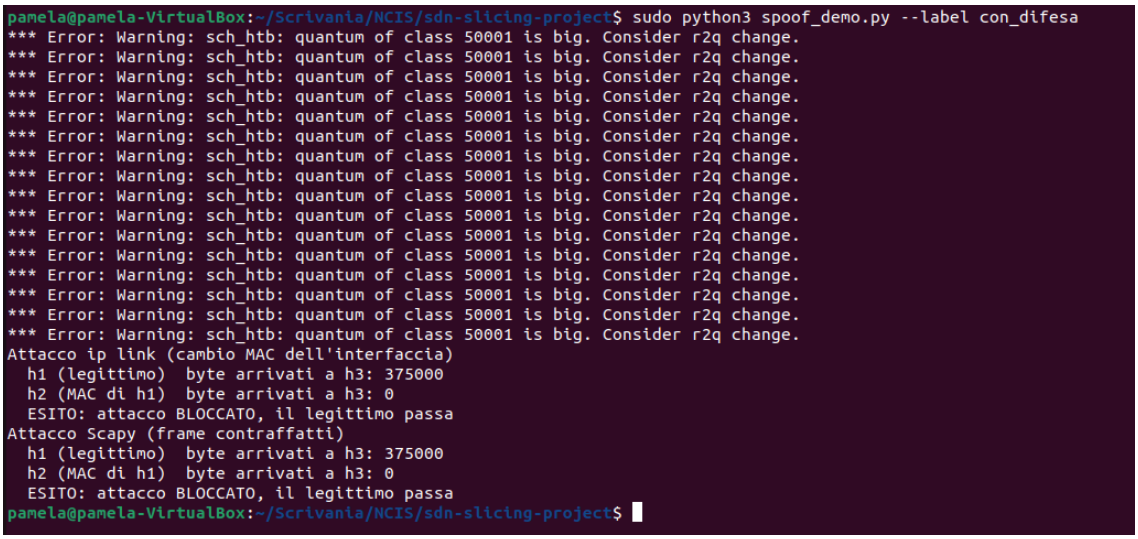


Figura 20: MAC spoofing con difesa: entrambi gli attacchi sono `BLOCCATI`, h2 arriva a 0 byte e il traffico legittimo di h1 (375000 byte) passa per intero. Le righe `sch_htb` in alto sono avvisi del kernel sulla configurazione delle code HTB e non influenzano il risultato.

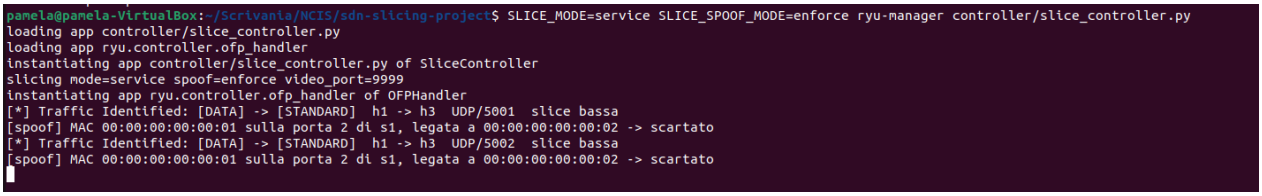
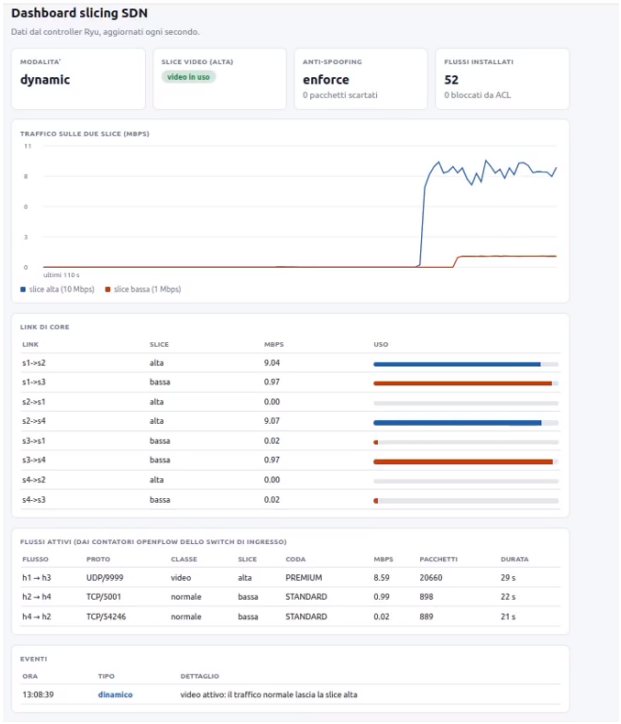


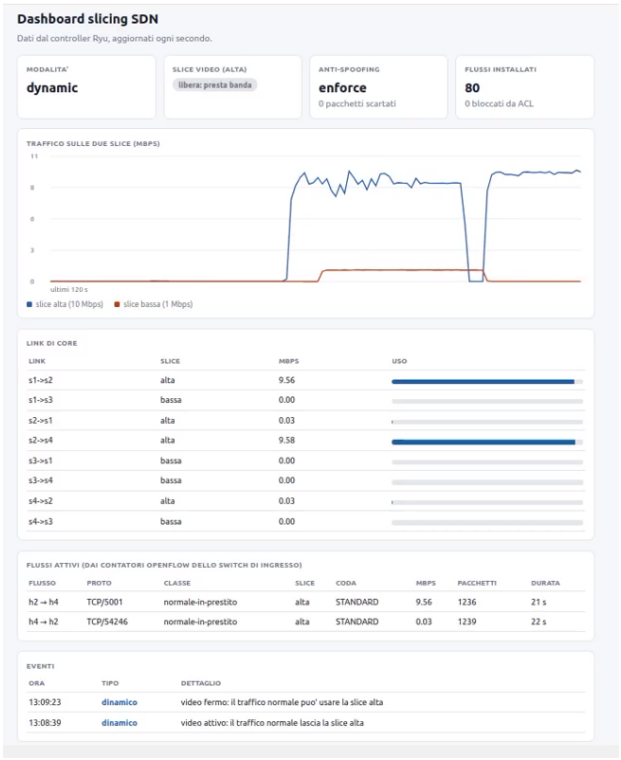
Figura 21: MAC spoofing: log del controller durante l'attacco. Ogni pacchetto con il MAC di h1 arrivato dalla porta 2 di `s1` (quella di h2) viene scartato.

8 Dashboard di monitoraggio

Il controller scrive ogni secondo, in modo atomico, un file `logs/state.json` con: la modalità, lo stato del video, i contatori (flussi installati, pacchetti scartati), la velocità di ogni link di core calcolata dai contatori di porta ($(\text{byte}_{\text{ora}} - \text{byte}_{\text{prima}}) \cdot 8/\Delta t$), l'andamento delle due slice, gli eventi e una **tabella dei flussi attivi** (chi parla con chi, protocollo e porta, classe, slice, coda, Mbps, pacchetti, durata) ricavata dalle statistiche dello switch di *ingresso* di ogni flusso, per non contare lo stesso flusso su tre switch. Un piccolo server HTTP (solo libreria standard di Python) serve a una pagina che interroga lo stato ogni secondo e disegna il grafico. Tutti i dati sono *misurati*, non simulati.



(a) Video attivo: slice alta a 8–9 Mbps, TCP normale sulla slice bassa.



(b) Slice libera: il TCP normale usa la slice alta.

Figura 22: Dashboard (dati misurati dai contatori OpenFlow di switch e porte).

9 Verifica e riproducibilità

Gli esperimenti in rete (topology, service, dynamic, spoofing) si lanciano con un controller in un terminale e uno script nell'altro:

```
SLICE_MODE=service ryu-manager controller/slice_controller.py     # terminale 1
sudo python3 run_experiment.py --mode service                      # terminale 2
```

e producono i risultati nella cartella `results/<modalità>/`: riepilogo, catture `tcpdump`, regole di ogni switch e, per lo slicing dinamico, il grafico. Gli script controllano le soglie attese e scrivono `ESITO: ok` oppure `ESITO: DA CONTROLLARE`.

10 Limiti e sviluppi futuri

- La rete è *statica e nota*: host, MAC e porte sono scritti nel codice, con un solo MAC per porta di accesso. Un'evoluzione naturale è la scoperta automatica della topologia (per esempio con LLDP).
- Lo slicing dinamico ha un ritardo di circa 4–5 s; si potrebbero usare i messaggi *flow removed* per ridurlo.
- Le misure sono state fatte in una VM con Mininet e ripetute due volte: indicano il comportamento *relativo* delle slice, non prestazioni assolute.

11 Conclusioni

È stato realizzato un controller Ryu che implementa il topology slicing e il service slicing su una rete a rombo con due slice da 10 e 1 Mbps, ed è stato esteso con slicing dinamico, anti-spoofing e

dashboard di monitoraggio.

Gli esperimenti, eseguiti e misurati in una macchina virtuale, mostrano che:

- ogni coppia ammessa usa davvero la propria slice, come verificato anche con `tcpdump`;
- il video riceve la banda offerta indipendentemente dal traffico normale, e il traffico normale resta intorno a 1 Mbps;
- la slice alta inutilizzata viene prestata al traffico normale e restituita al video con un ritardo di pochi secondi;
- i pacchetti con MAC falsificato sono scartati senza danneggiare gli host legittimi.